

[Linux Fundamentals & Investigation Notes]

[Linux File System Structure]

[What is the Linux File System?]

Linux organizes everything in a single hierarchical tree structure. Unlike Windows, which uses separate drives like C:\, D:\, or E:\, Linux starts everything from a single point represented by a forward slash /. This is known as the **Root Directory**.

```
/ (Root)
├── bin
├──/sbin
├── etc
├── dev
├── home
├── root
├── usr
│   ├── bin
│   ├── include
│   ├── lib
│   ├── local
│   └── share
├── tmp
├── var
│   ├── cache
│   ├── log
│   ├── spool
│   └── tmp
├── boot
├── lib
└── opt
```

[Directory Hierarchy & Meanings]

- **/bin**: Essential user command binaries (e.g., bash, cat, chmod, cp, date, echo, grep, hostname, kill, ls, mkdir, nano, ping, ps, pwd, rm, tar, touch).
- **/sbin**: Essential system binaries used for administration (e.g., fdisk, fsck, halt, ifconfig, init, mkfs, reboot, route).
- **/etc**: Configuration files for the system (e.g., crontab, fstab, hosts, resolv.conf, services).
- **/dev**: Device files that represent hardware devices (includes special files like /dev/null).
- **/home**: Home directories for regular users.
- **/root**: The home directory specifically for the root (administrator) user.
- **/usr**: Read-only user applications, support data, and binaries.
 - **/usr/bin**: Most user commands.
 - **/usr/include**: Standard include files for 'C' code.
 - **/usr/lib**: Object, binary, and library files for coding and packages.
 - **/usr/local**: Locally installed software.
 - **/usr/share/man**: Manual pages.
- **/tmp**: Temporary files.
- **/var**: Variable data files that change constantly as the system runs.
 - **/var/cache**: Application cache data.

- **/var/log**: System and application log files.
- **/var/spool**: Tasks waiting to be processed (e.g., cron jobs, mail).
- **/var/tmp**: Temporary files saved between reboots.
- **/boot**: Boot loader files and kernel images.
- **/lib**: System libraries and kernel modules.
- **/opt**: Optional add-on application software packages.
- **/mnt**: Mount points for temporary file systems.
- **/media**: Removable media (like USB drives).
- **/srv**: Service data.
- **/sys**: Kernel-related information (similar to /proc, but more detailed).
- **/proc**: A virtual filesystem containing information about the system, process information, and network status.

[Linux Permissions]

In Linux, everything is considered a file, and every file is assigned an owner and a group. Permissions dictate how the Owner, the Group, and Everyone Else can interact with these files.

[Permission Types & Meanings]

Permission	Symbol	File Meaning	Directory Meaning
Read	r	View the file contents.	List the files inside using ls.
Write	w	Modify the file contents.	Create, delete, or rename files inside.
Execute	x	Run the file as a program.	"Enter" the directory using cd.

Note: To delete a file, you must have Write and Execute permissions on its parent directory, not necessarily on the file itself.

[Dissecting Permission Strings]

When running the `ls -l` command, permissions appear as a 10-character string (e.g., `-rwxr-xr--`).

1. **First Character**: Defines the file type (- for a regular file, d for a directory, l for a symbolic link).
2. **Next Nine Characters**: Divided into three sets of triplets:
 - **User (u)**: The individual who owns the file.
 - **Group (g)**: Members of the group assigned to the file.
 - **Others (o)**: Everyone else on the system.

Example Analysis (-rwxr-xr--):

- **User**: rwx (Read, Write, Execute)
- **Group**: r-x (Read, Execute)
- **Others**: r-- (Read only)

[Numeric (Octal) Notation]

System administrators often use a base-8 system to assign permissions mathematically:

- **Read (r)** = 4
- **Write (w)** = 2
- **Execute (x)** = 1

- **None (-) = 0**

By adding up the values in each triplet, you get a single number. For example, `rwX` is $4 + 2 + 1 = 7$.

Total Permissions	Octal Notation	Description
7	rwX	Full control
6	rw-	Read and write
5	r-X	Read and execute
4	r--	Read-only
0	---	No access

- **Common Command Example:** `chmod 644 file.txt` sets the file to Read/Write for the owner, and Read-only for everyone else.

[Linux Logging (/var/log)]

The `/var/log` directory is the main logging hub in Linux, containing authentication, system, and application logs. System investigations rely heavily on log analysis within this directory.

[Important Log Files]

- **/var/log/auth.log:** Tracks authentication events including SSH logins, failed login attempts, and sudo usage (primarily used in Ubuntu/Debian).
- **/var/log/secure:** The RedHat/CentOS equivalent to `auth.log`, tracking user authentication and security events.
- **/var/log/kern.log:** Contains system kernel logs.
- **/var/log/syslog:** Contains general system logs.
- **/var/log/messages:** Logs general system activity.
- **/var/log/apache2/access.log:** Tracks web access logs.
- **/var/log/apache2/error.log:** Tracks web server errors.

Example Authentication Log Entry:

```
Jul 10 10:22:11 server sshd[2345]: Accepted password for admin from 192.168.1.10
```

[SSH (Secure Shell) & Authentication]

[What is SSH?]

SSH provides secure, encrypted remote access to Linux machines over **Port 22**. Attackers target SSH because it allows remote control, persistence, and lateral movement across networks.

[Key Components]

- **ssh:** The client-side command line tool.
- **sshd:** The SSH daemon or server process running continuously to listen for incoming connections.
- **Port 22:** The default port configuration.

[SSH Authentication Methods]

1. Password Authentication

- The client sends the username and password over an encrypted tunnel.
- The server validates these credentials against the /etc/shadow file.
- *Drawback*: Simple to set up but highly vulnerable to brute-force attacks.

2. Public Key Authentication (Asymmetric Encryption)

This method utilizes key pairs (RSA or Ed25519) and is the professional standard for security.

- **The Setup:** A **Private Key** remains safely on your local machine, while the corresponding **Public Key** is stored on the server inside the `~/.ssh/authorized_keys` file.
- **The Challenge-Response Process:**
 1. The client connects and specifies which key it wants to authenticate with.
 2. The server generates a random number challenge, encrypts it with the user's Public Key, and sends it back to the client.
 3. The client decrypts the challenge using its Private Key, combines it with the current session ID, and sends back a cryptographic signature.
 4. The server verifies this signature. If authenticated successfully, access is granted.
- **Configuration File Location:** `/etc/ssh/sshd_config`

[Technical Image Architecture]

```
[ Application Layer ]
| Command Execution: ssh username@host
▼
[ Transport Layer (ssh-client) ]
| Contains: Username, Host, Private Key, Message
|
▼ (Message sent in the form of Packets)
|
[ Transport Layer (ssh-server) ]
| Verification: Private key gets verified by matching
| keys present in ~/.ssh/authorized_keys
▼
[ User Authentication Layer ]
| (Once Private Key is Verified)
▼
Allows commands to run on the ssh-server terminal from the client,
sends/receives files, and initiates secure network tunneling.
```

[Investigation Flow & Investigation Mindset]

[Common Compromise Sequence]

When auditing systems for unauthorized access, malicious events typically follow a noticeable pattern:

```
graph LR
    A[Brute-Force Attack] --> B[Successful Compromise]
    B --> C[Privilege Escalation]
```

- **Analytical Interpretation:** A Brute-Force attack led to a successful Compromise, which was immediately followed by Privilege Escalation. Attackers frequently plant their own keys in `authorized_keys` files to maintain persistence on the host.

[Analytical Questions to Ask]

When auditing an incident on Linux systems, do not simply ask *"Was there a login?"*. Instead, answer these precise questions:

- Who logged in?
- From what location or IP address?
- What authentication method was utilized?
- What specific actions were executed post-login?
- Was there any attempt or success at privilege escalation?